



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Experiment No. 8
Apply FP based estimation technique for a case study
Date of Performance: 11/09/23
Date of Submission: 25/09/23



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: To apply FP based estimation technique for a case study.

Objective : To analyse and apply FP Based estimation technique for Hotel Management

Theory:

Function Point (FP) is an element of software development which helps to approximate the cost of development early in the process. It may measure functionality from user's point of view.

System

Objectives of FPA:

- The objective of FPA is to measure the functionality that the user requests and receives.
- The objective of FPA is to measure software development and maintenance independently of the technology used for implementation.
- It should be simple enough to minimize the overhead of the measurement process.
- It should be a consistent measure among various projects and organizations.

Types of FPA:

•

information that

Transactional Functional Type –

- **External Input (EI):** EI processes data or control
 - comes from outside the application's boundary. The EI is an elementary process.
- **External Output (EO):** EO is an elementary process that generates data or control information sent outside the application's boundary.
- **External Inquiries (EQ):** EQ is an elementary process made up of an input-output combination that results in data retrieval.

Data Functional Type –

- **Internal Logical File (ILF):** A user identifiable group of logically related data or control information maintained within the boundary of the application.
- **External Interface File (EIF):** A group of users recognizable logically related data allusion to the software but maintained within the boundary of another software.

•

STEPS:

Step-1:

$F = 14 * \text{scale}$

Scale varies from 0 to 5 according to character of Complexity Adjustment Factor (CAF). Below table shows scale:



- 0 - No Influence
- 1 - Incidental
- 2 - Moderate
- 3 - Average
- 4 - Significant
- 5 - Essential
- **Step-2:** Calculate Complexity Adjustment Factor (CAF).
 $CAF = 0.65 + (0.01 * F)$
- **Step-3:** Calculate Unadjusted Function Point (UFP).
Multiply each individual function point to corresponding values in TABLE.
- **Step-4:** Calculate Function Point.
 $FP = UFP * CAF$

Example:

Given the following values, compute function point when all complexity adjustment factor (CAF) and weighting factors are average.

User Input = 50

User Output = 40

User Inquiries = 35

User Files = 6

External Interface = 4

Explanation:

- **Step-1:** As complexity adjustment factor is average (given in question), hence, scale = 3.
- $F = 14 * 3 = 42$
- **Step-2:**
 $CAF = 0.65 + (0.01 * 42) = 1.07$
- **Step-3:** As weighting factors are also average (given in question) hence we will multiply each individual function point to corresponding values in TABLE.
 $UFP = (50*4) + (40*5) + (35*4) + (6*10) + (4*7) = 628$
- **Step-4:**
Function Point = $628 * 1.07 = 671.96$



Solution:

FP CODE:

```
#include <iostream>
using namespace std;

// Function to calculate Function Point
void calfp()
{
    int frates[5][3];
    double CAF;

    cout << "Enter Function Rates for the following Function Units:" << endl;

    // Prompt user to enter function rates
    string funUnits[5] = {
        "External Inputs",
        "External Outputs",
        "External Inquiries",
        "Internal Logical Files",
        "External Interface Files"
    };

    string wtRates[3] = { "Low", "Average", "High" };

    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 3; j++) {
            cout << "Enter rate for " << wtRates[j] << " for " << funUnits[i] << ": ";
            cin >> frates[i][j];
        }
    }

    cout << "Enter the Complexity Adjustment Factor (CAF) rating: ";
    cin >> CAF;

    int wtFactors[5][3] = {
        { 3, 4, 6 },
        { 4, 5, 7 },
```



```
{ 3, 4, 6 },
{ 7, 10, 15 },
{ 5, 7, 10 },
};

int UFP = 0;

// Calculating UFP (Unadjusted Function Point)
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 3; j++) {
        int freq = frates[i][j];
        UFP += freq * wtFactors[i][j];
    }
}

// Calculate Function Point (FP)
double FP = UFP * CAF;

// Output Values
cout << "Function Point Analysis :-" << endl;
cout << "Unadjusted Function Points (UFP) : " << UFP << endl;
cout << "Complexity Adjustment Factor (CAF) : " << CAF << endl;
cout << "Function Points (FP) : " << FP << endl;
}

// Driver function
int main()
{
    calfp();
    return 0;
}
```

OUTPUT:

Enter Function Rates for the following Function Units:

Enter rate for Low for External Inputs: 0

Enter rate for Average for External Inputs: 0

Enter rate for High for External Inputs: 10

Enter rate for Low for External Outputs: 0



Enter rate for Average for External Outputs: 0

Enter rate for High for External Outputs: 7

Enter rate for Low for External Inquiries: 0

Enter rate for Average for External Inquiries: 0

Enter rate for High for External Inquiries: 5

Enter rate for Low for Internal Logical Files: 0

Enter rate for Average for Internal Logical Files: 0

Enter rate for High for Internal Logical Files: 3

Enter rate for Low for External Interface Files: 0

Enter rate for Average for External Interface Files: 0

Enter rate for High for External Interface Files: 2

Enter the Complexity Adjustment Factor (CAF) rating: 1.17

Function Point Analysis :-

Unadjusted Function Points (UFP) : 204

Complexity Adjustment Factor (CAF) : 1.17

Function Points (FP) : 238.68

Conclusion: